

```
vgpreload_helgrind-amd64-linux.so)
==16932== by 0x4E42DC4: start_thread (in /usr/lib64/
libpthread-2.17.so)
==16932== by 0x514D21C: clone (in /usr/lib64/libc-2.17.so)
==16932==
==16932== This conflicts with a previous write of size 4 by
thread #2
==16932== Locks held: none
==16932== at 0x400829: inc_counter (race.c:23)
==16932== by 0x4C2F881: ??? (in /usr/lib64/valgrind/
vgpreload_helgrind-amd64-linux.so)
==16932== by 0x4E42DC4: start_thread (in /usr/lib64/
libpthread-2.17.so)
==16932== by 0x514D21C: clone (in /usr/lib64/libc-2.17.so)
==16932== Address 0x601084 is 0 bytes inside data symbol
"shared_counter"
```

Here it indicates a conflict between [thread #3, line 16] and [thread #2, line23]. Let's check the source and analyse it. The following race condition occurs:

- Thread #3 acquires the mutex lock and reads the shared variable. This is evident from line 16 of the source.
 - At the same time, Thread #2 is writing back to the shared variable as evident from line 23. This is a race situation.
4. Now, we come to the second race situation:

```
==16932== Possible data race during write of size 4 at
0x601084 by thread #3
==16932== Locks held: none
==16932== at 0x400829: inc_counter (race.c:23)
==16932== by 0x4C2F881: ??? (in /usr/lib64/valgrind/
vgpreload_helgrind-amd64-linux.so)
==16932== by 0x4E42DC4: start_thread (in /usr/lib64/
libpthread-2.17.so)
==16932== by 0x514D21C: clone (in /usr/lib64/libc-2.17.so)
==16932==
==16932== This conflicts with a previous write of size 4 by
thread #2
==16932== Locks held: none
==16932== at 0x400829: inc_counter (race.c:23)
==16932== by 0x4C2F881: ??? (in /usr/lib64/valgrind/
vgpreload_helgrind-amd64-linux.so)
==16932== by 0x4E42DC4: start_thread (in /usr/lib64/
libpthread-2.17.so)
==16932== by 0x514D21C: clone (in /usr/lib64/libc-2.17.so)
==16932== Address 0x601084 is 0 bytes inside data symbol
"shared_counter"
```

- Here it indicates the conflict between [thread #3, line 23] and [thread #2, line23]. Upon analysing the source, it is evident that both Thread #2 and Thread #3 can write to the shared variable at the same time as they are not using any

locking mechanism.

- To fix both the race scenarios, the mutex should be acquired both before reading and before writing the shared counter. In the source we can see that mutex is acquired before reading but not while writing the shared counter. The following change in the source fixes the race condition:

```
.....
.....
pthread_mutex_lock(&mutex);
// Write back to the shared variable
shared_counter = current_value;
pthread_mutex_unlock(&mutex);
.....
.....
```

- After fixing, if we again run the program with Helgrind, we get the following output, which suggests that the race condition has been fixed.

```
$ valgrind --tool=helgrind ./race_fixed
==17034== Helgrind, a thread error detector
==17034== Copyright (C) 2007-2013, and GNU GPL'd, by
OpenWorks LLP et al.
==17034== Using Valgrind-3.10.0 and LibVEX; rerun with -h for
copyright info
==17034== Command: ./race
==17034==
Final value of shared counter: 20000
==17034==
==17034== For counts of detected and suppressed errors, rerun
with: -v
==17034== Use --history-level=approx or =none to gain
increased speed, at
==17034== the cost of reduced accuracy of conflicting-access
information
==17034== ERROR SUMMARY: 0 errors from 0 contexts
(suppressed: 60005 from 11)
```

Helgrind is a valuable tool for detecting deadlocks and race conditions in a multithreaded project, leading to reliable and robust multithreaded applications. By analysing thread synchronisation issues, it helps developers identify potential issues early in the development process. **END** 🐧

 By: Subranshu Patel

The author is group manager in the system software division of HCL Technologies Limited. He has extensive experience in Linux-based OS and ecosystem design and development, and in open source technologies.